Summarize

Chapter 3. Splitting the Monolith

Many of you reading this book likely don't have a blank slate on which to design your system, and even if you did, starting with microservices might not be a great idea, for reasons we explored in Chapter 1. Many of you will already have an existing system, perhaps some form of monolithic architecture, which you are looking to migrate to a microservice architecture.

In this chapter I'll outline some first steps, patterns, and general tips to help you navigate the transition to a microservice architecture.

Have a Goal

Microservices are not the goal. You don't "win" by having microservices. Adopting a microservice architecture should be a conscious decision, one based on rational decision making. You should be thinking of migrating to a microservice architecture only if you can't find any easier way to move toward your end goal with your current architecture.

Without a clear understanding as to what you are trying to achieve, you could fall into the trap of confusing activity with outcome. I've seen teams obsessed with creating microservices without ever asking why. This is problematic in the extreme given the new sources of complexity that microservices can introduce.

Fixating on microservices rather than on the end goal also means you will likely stop thinking of other ways in which you might bring about the change you are looking for. For example, microservices can help you scale your system, but there are often a number of alternative scaling techniques that should be looked at first. Spinning up a few more copies of your existing monolithic system behind a load balancer may well help you scale your system much more effectively than going through a complex and lengthy decomposition to microservices.

TIP

Microservices aren't easy. Try the simple stuff first.

Finally, without a clear goal, it becomes difficult to know where to start. Which microservice should you create first? Without an overarching understanding of what you are trying to achieve, you're flying blind.

So be clear about what change you are trying to achieve, and consider easier ways to achieve that end goal before considering microservices. If microservices really are the best way to move forward, then track your progress against that end goal and change course as necessary.

Incremental Migration

> *If you do a big-bang rewrite, the only thing you're guaranteed of is a big bang.*
>
> —Martin Fowler

If you get to the point of deciding that breaking apart your existing monolithic system is the right thing to do, I strongly advise you to chip away at the monolith, extracting a bit at a time. An incremental approach will help you learn about microservices as you go and will also limit the impact of getting something wrong (and you will get things wrong!). Think of our monolith as a block of marble. We could blow the whole thing up, but that rarely ends well. It makes much more sense to just chip away at it incrementally.

Break the big journey into lots of little steps. Each step can be carried out and learned from. If it turns out to be a retrograde step, it was only a small one. Either way, you learn from it, and the next step you take will be informed by those steps that came before.

Breaking things into smaller pieces also allows you to identify quick wins and learn from them. This can help make the next step easier and can help build momentum. By splitting out microservices one at a time, you also get to unlock the value they bring incrementally, rather than having to wait for some big bang deployment.

All of this leads to what has become my stock advice for people looking at microservices: if you think microservices are a good idea, start somewhere small. Choose one or two areas of functionality, implement them as microservices, get them deployed into produc-

tion, and then reflect on whether creating your new microservices helped you get closer to your end goal.

WARNING

You won't appreciate the true horror, pain, and suffering that a microservice architecture can bring until you are running in production.

The Monolith Is Rarely the Enemy

While I already made the case at the start of the book that some form of monolithic architecture can be a totally valid choice, it warrants repeating that a monolithic architecture isn't inherently *bad* and therefore shouldn't be viewed as the enemy. Don't focus on "not having the monolith"; focus instead on the benefits you expect your change in architecture to bring.

It is common for the existing monolithic architecture to remain after a shift toward microservices, albeit often in a diminished capacity. For example, a move to improve the ability of the application to handle more load might be satisfied by removing the 10% of functionality that is currently bottlenecked, leaving the remaining 90% in the monolithic system.

Many people find the reality of a monolith and microservices coexisting to be "messy"—but the architecture of a real-world running system is never clean or pristine. If you want a "clean" architecture, by all means laminate a printout of an idealized version of the system architecture you might have had, if only you had perfect foresight and limitless funds. Real system architecture is a constantly evolving thing that must adapt as needs and knowledge change. The skill is in getting used to this idea, something I'll come back to in Chapter 16.

By making your migration to microservices an incremental journey, you are able to chip away at the existing monolithic architecture, delivering improvements along the way, while also, importantly, knowing when to stop.

In surprisingly rare circumstances, the demise of the monolith might be a hard requirement. In my experience, this is often limited to situations in which the existing monolith is based on dead or dying technology, is tied to infrastructure that needs to be retired, or is perhaps an expensive third-party system that you want to ditch. Even in these situa-

tions, an incremental approach to decomposition is warranted for the reasons I've outlined.

The Dangers of Premature Decomposition

There is danger in creating microservices when you have an unclear understanding of the domain. An example of the problems this can cause comes from my previous company, Thoughtworks. One of its products was Snap CI, a hosted continuous integration and continuous delivery tool (we'll discuss those concepts in Chapter 7). The team had previously worked on a similar tool, GoCD, a now open source continuous delivery tool that can be deployed locally rather than being hosted in the cloud.

Although there was some code reuse very early on between the Snap CI and GoCD projects, in the end Snap CI turned out to be a completely new codebase. Nonetheless, the previous experience of the team in the domain of CD tooling emboldened them to move more quickly in identifying boundaries and building their system as a set of microservices.

After a few months, though, it became clear that the use cases of Snap CI were subtly different enough that the initial take on the service boundaries wasn't quite right. This led to lots of changes being made across services, and an associated high cost of change. Eventually, the team merged the services back into one monolithic system, giving the team members time to better understand where the boundaries should exist. A year later, the team was able to split the monolithic system into microservices, whose boundaries proved to be much more stable. This is far from the only example of this situation I have seen. Prematurely decomposing a system into microservices can be costly, especially if you are new to the domain. In many ways, having an existing codebase you want to decompose into microservices is much easier than trying to go to microservices from the beginning for this very reason.

What to Split First?

Once you have a firm grasp on *why* you think microservices are a good idea, you can use this understanding to help prioritize which microservices to create first. Want to scale the application? Functionality that currently constrains the system's ability to handle load is going to be high on the list. Want to improve time to market? Look at the system's volatility to identify those pieces of functionality that change most frequently, and see if they would work as microservices. You can use static analysis tools like CodeScene to quickly find volatile parts of your codebase. You can see an example of a

view from CodeScene in Figure 3-1, where we see hotspots in the open source Apache Zookeeper project.

But you also have to consider what decompositions are viable. Some functionality can be so deeply baked into the existing monolithic application that it is impossible to see how it can be detangled. Or perhaps the functionality in question is so critical to the application that any changes are considered high risk. Alternatively, the functionality you want to migrate might already be somewhat self-contained, and so the extraction seems very straightforward.

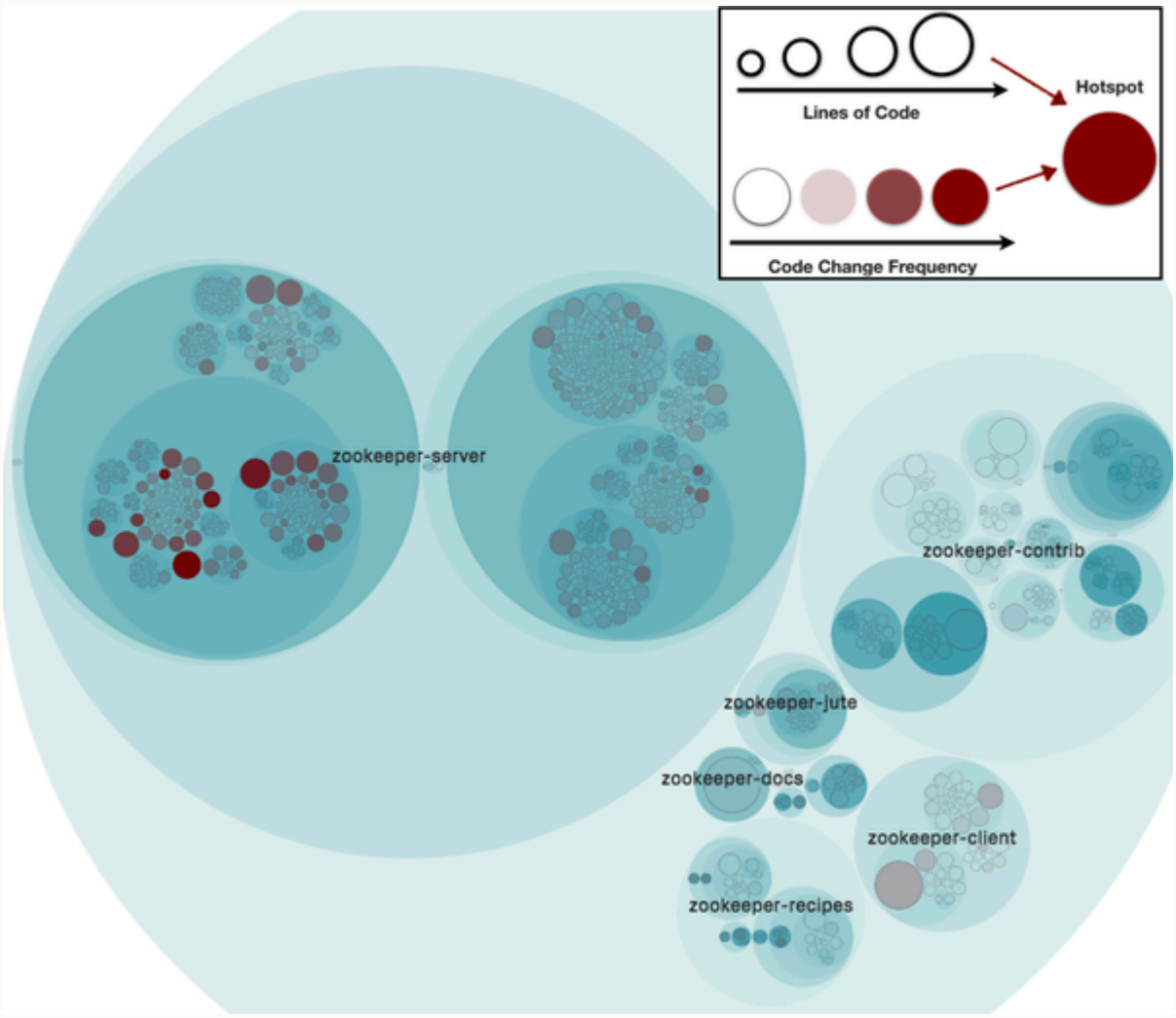


Figure 3-1. The hotspot view in CodeScene, helping identify parts of the codebase that change frequently

Fundamentally, the decision about which functionality to split into a microservice will end up being a balance between these two forces—how easy the extraction is versus the benefit of extracting the microservice in the first place.

My advice for the first couple of microservices would be to pick things that lean a bit more toward the "easy" end of the spectrum—a microservice that we think has some impact in terms of achieving our end-to-end goal, certainly, but something we'd consider to be low-hanging fruit. It is important with a transition like this, especially one that

could take months or years, to gain a sense of momentum early on. So you need some quick wins under your belt.

On the other hand, if you try to extract what you consider to be the easiest microservice and aren't able to make it work, it might be worth reconsidering whether microservices are really right for you and your organization.

With a few successes and some lessons learned, you'll be much better placed to tackle more complex extractions, which may also be operating in more critical areas of functionality.

Decomposition by Layer

So you've identified your first microservice to extract; what next? Well, we can break that decomposition down into further, smaller steps.

If we consider the traditional three tiers of a web-based services stack, then we can look at the functionality we want to extract in terms of its user interface, backend application code, and data.

The mapping from a microservice to a user interface is often not 1:1 (this is a topic we explore in a lot more depth in Chapter 14). As such, extracting user interface functionality related to the microservice could be considered a separate step. I will sound a note of caution here about ignoring the user interface part of the equation. I've seen far too many organizations look only at the benefits of decomposing the backend functionality, which often results in an overly siloed approach to any architectural restructuring. Sometimes the biggest benefits can come from decomposition of the UI, so ignore this at your peril. Often decomposition of the UI tends to lag behind decomposition of the backend into microservices, since until the microservices are available, it's difficult to see the possibilities for UI decomposition; just make sure it doesn't lag too much.

If we then look at the backend code and related storage, it's vital for both to be in scope when extracting a microservice. Let's consider Figure 3-2, where we are looking to extract functionality related to managing a customer's wishlist. There is some application code that lives in the monolith, and some related data storage in the database. So which bit should we extract first?

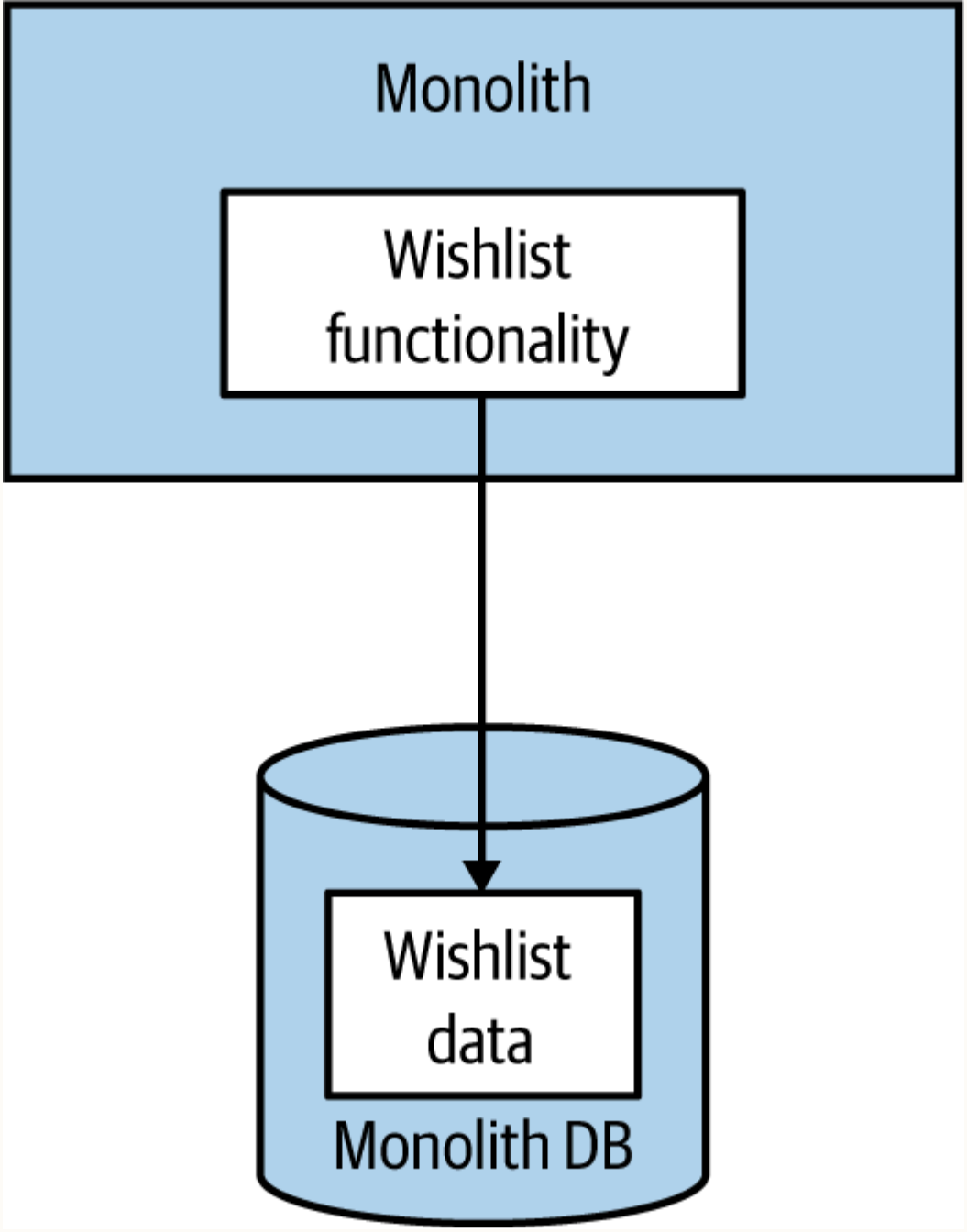


Figure 3-2. The wishlist code and data in the existing monolithic application

Code First

In Figure 3-3, we have extracted the code associated with the wishlist functionality into a new microservice. The data for the wishlist remains in the monolithic database at this stage—we haven't completed the decomposition until we've also moved out the data related to the new `Wishlist` microservice.

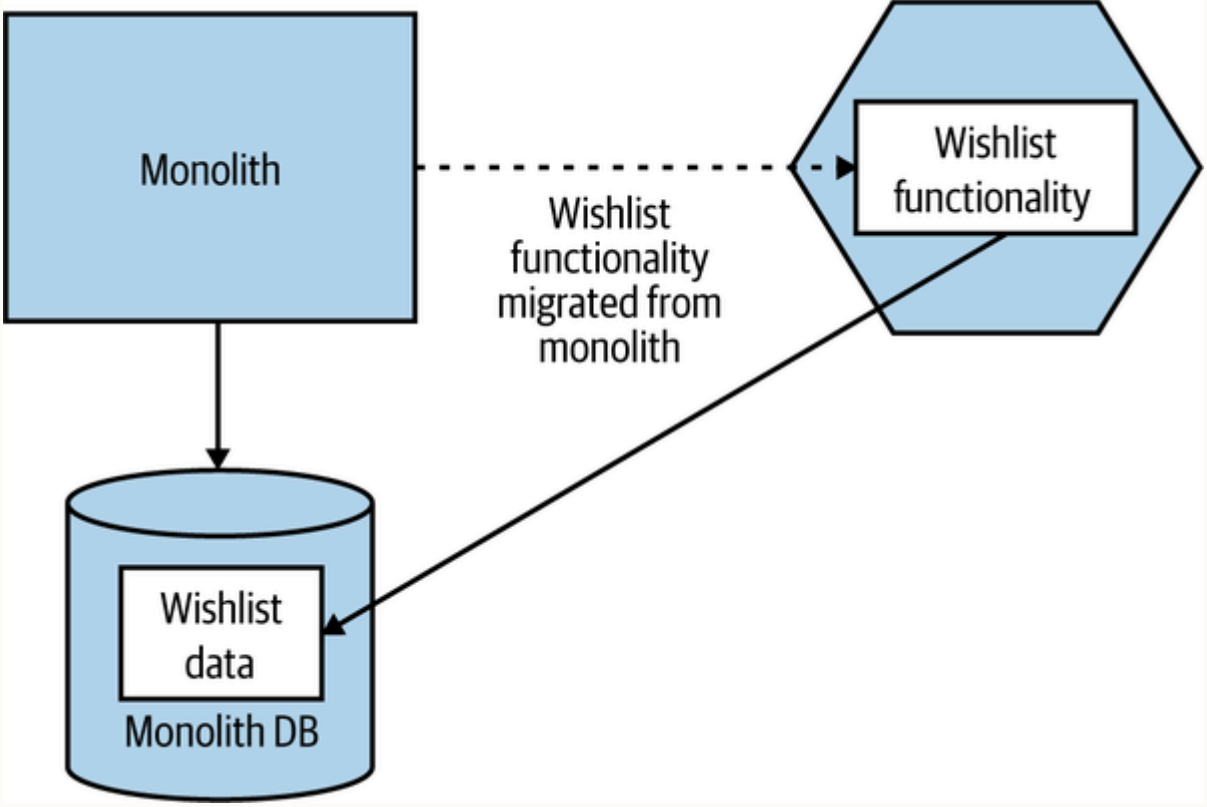


Figure 3-3. Moving the wishlist code into a new microservice first, leaving the data in the monolithic database

In my experience, this tends to be the most common first step. The main reason for this is that it tends to deliver more short-term benefit. If we left the data in the monolithic database, we're storing up lots of pain for the future, so that does need to be addressed too, but we have gained a lot from our new microservice.

Extracting the application code tends to be easier than extracting things from the database. If we found that it was impossible to extract the application code cleanly, we could abort any further work, avoiding the need to detangle the database. If, however, the application code is cleanly extracted but extracting the data proves to be impossible, we could be in trouble—thus it's essential that even if you decide to extract the application code before the data, you need to have looked at the associated data storage and have some idea as to whether extraction is viable and how you will go about it. So do the legwork to sketch out how both application code and data will be extracted before you start.

Data First

In Figure 3-4, we see the data being extracted first, before the application code. I see this approach less often, but it can be useful in situations in which you are unsure whether the data can be separated cleanly. Here, you prove that this can be done before moving on to the hopefully easier application code extraction.

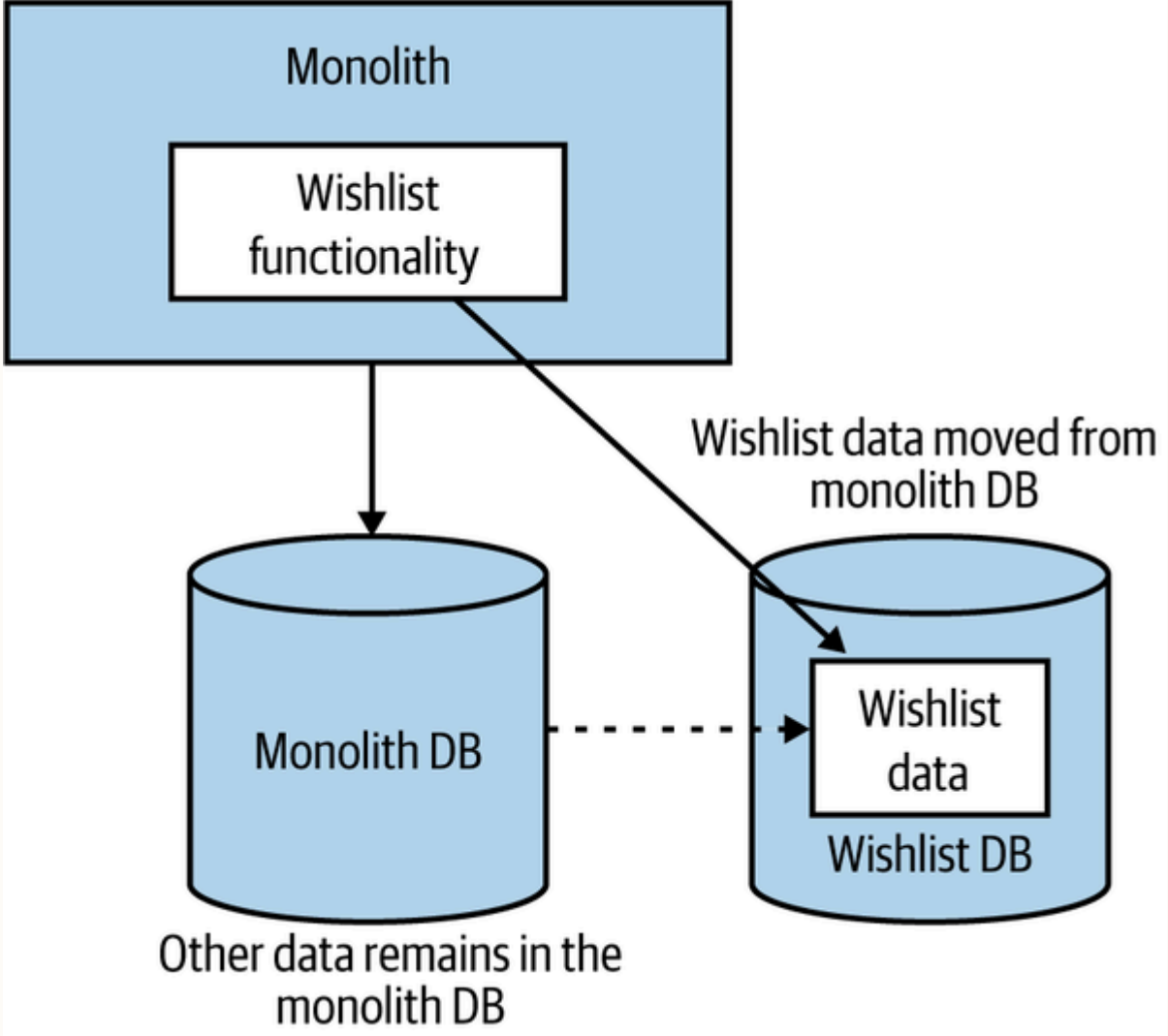


Figure 3-4. The tables associated with the wishlist functionality are extracted first

The main benefit of this approach in the short term is in derisking the full extraction of the microservice. It forces you to deal up front with issues like loss of enforced data integrity in your database or lack of transactional operations across both sets of data. We'll touch briefly on the implications of both issues later in this chapter.

Useful Decompositional Patterns

A number of patterns can be useful in helping break apart an existing system. Many of these are explored in detail in my book *Monolith to Microservices*;[1] rather than repeat them all here, I will share an overview of some of them to give you an idea of what is possible.

Strangler Fig Pattern

A technique that has seen frequent use during system rewrites is the *strangler fig pattern*, a term coined by Martin Fowler. Inspired by a type of plant, the pattern describes the process of wrapping an old system with the new system over time, allowing the new system to take over more and more features of the old system incrementally.

The approach as shown in Figure 3-5 is straightforward. You intercept calls to the existing system—in our case the existing monolithic application. If the call to that piece of functionality is implemented in our new microservice architecture, it is redirected to the microservice. If the functionality is still provided by the monolith, the call is allowed to continue to the monolith itself.

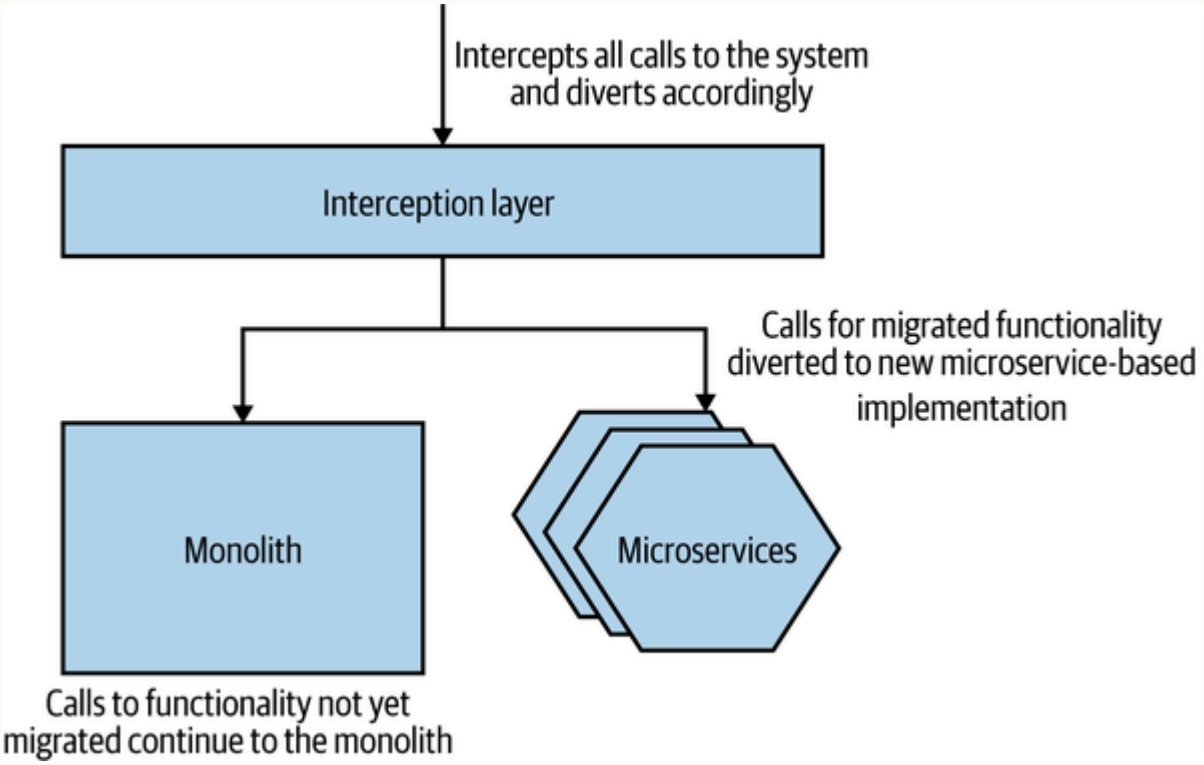


Figure 3-5. An overview of the strangler fig pattern

The beauty of this pattern is that it can often be done without making any changes to the underlying monolithic application. The monolith is unaware that it has even been "wrapped" with a newer system.

Parallel Run

When switching from functionality provided by an existing tried and tested application architecture to a fancy new microservice-based one, there may be some nervousness, especially if the functionality being migrated is critical to your organization.

One way to make sure the new functionality is working well without risking the existing system behavior is to make use of the parallel run pattern: running both your monolithic implementation of the functionality and the new microservice implementation side by side, serving the same requests, and comparing the results. We'll explore this pattern in more detail in "Parallel Run".

Feature Toggle

A feature toggle is a mechanism that allows a feature to be switched off or on, or to switch between two different implementations of some functionality. The feature toggle is a pattern that has good general applicability, but it can be especially useful as part of a microservice migration.

As I outlined with the strangler fig application, we'll often leave the existing functionality in place in the monolith during the transition, and we'll want the ability to switch between versions of the functionality—the functionality in the monolith and that in the new microservice. With the strangler fig pattern example of using an HTTP proxy, we could implement the feature toggle in the proxy layer to allow for a simple control to switch between implementations.

For a broader introduction to feature toggles, I recommend Pete Hodgson's article "Feature Toggles (aka Feature Flags)."[2]

Data Decomposition Concerns

When we start breaking databases apart, we can cause a number of issues. Here are a few of the challenges you might face, and some tips to help.

Performance

Databases, especially relational databases, are good at joining data across different tables. Very good. So good, in fact, that we take this for granted. Often, though, when we split databases apart in the name of microservices, we end up having to move join operations from the data tier up into the microservices themselves. And try as we might, it's unlikely to be as fast.

Consider Figure 3-6, which illustrates a situation we find ourselves in regarding MusicCorp. We've decided to extract our catalog functionality—something that can manage and expose information about artists, tracks, and albums. Currently, our catalog-related code inside the monolith uses an `Albums` table to store information about the CDs that we might have available for sale. These albums end up getting referenced in our `Ledger` table, which is where we track all sales. The rows in the `Ledger` table record the date on which something is sold, along with an identifier that refers to the item sold; the identifier in our example is called a SKU (stock keeping unit), a common practice in retail systems.[3]

At the end of each month, we need to generate a report outlining our best-selling CDs. The `Ledger` table helps us understand which SKU sold the most copies, but the information about that SKU is over in the `Albums` table. We want to make the reports nice and easy to read, so rather than saying, "We sold 400 copies of SKU 123 and made $1,596," we'd add more information about what was sold, saying instead, "We sold 400 copies of *Now That's What I Call Death Polka* and made $1,596." To do this, the database query triggered by our finance code needs to join information from the `Ledger` table to the `Albums` table, as Figure 3-6 shows.

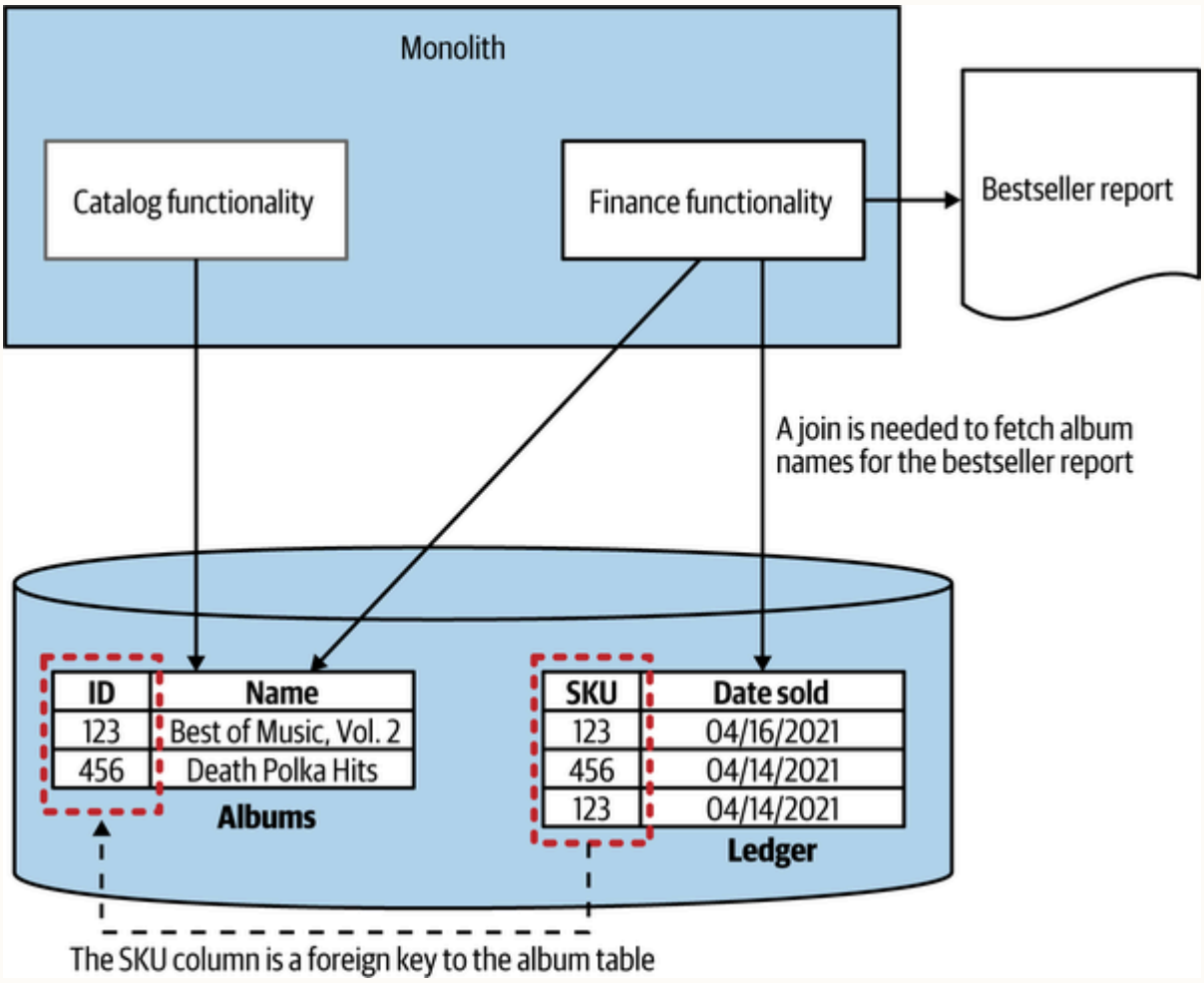


Figure 3-6. A join operation in the monolithic database

In our new microservice-based world, our new `Finance` microservice has the responsibility of generating the bestsellers report but doesn't have the album data locally. So it will need to fetch this data from our new `Catalog` microservice, as shown in Figure 3-7. When generating the report, the `Finance` microservice first queries the `Ledger` table, extracting the list of best-selling SKUs for the last month. At this point, the only information we have locally is a list of SKUs and the number of copies sold for each SKU.

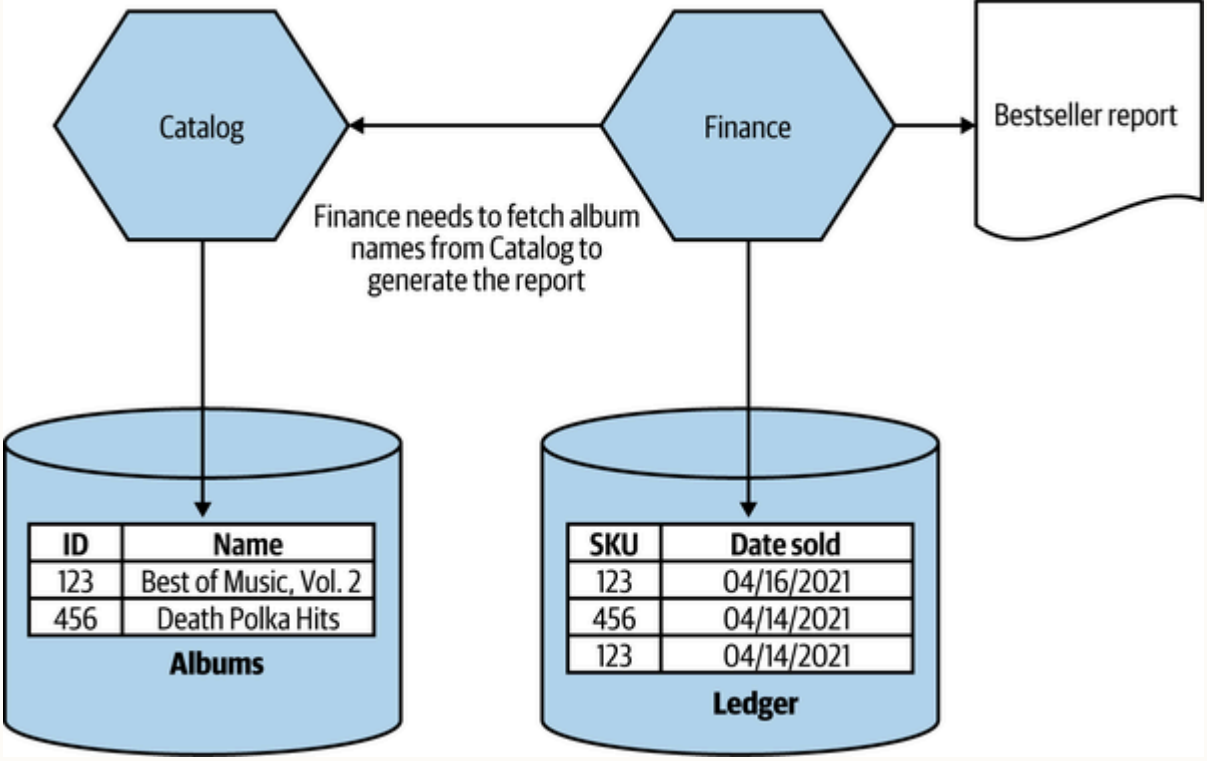


Figure 3-7. Replacing a database join operation with service calls

Next, we need to call the `Catalog` microservice, requesting information on each of these SKUs. This request in turn will cause the `Catalog` microservice to make its own local `SELECT` on its own database.

Logically, the join operation is still happening, but it is now happening inside the `Finance` microservice rather than in the database. The join has gone from the data tier to the application code tier. Unfortunately, this operation isn't going to be anywhere near as efficient as it would have been had the join remained in the database. We've gone from a world in which we have a single `SELECT` statement, to a new world in which we have a `SELECT` query against the `Ledger` table, followed by a call to the `Catalog` microservice, which in turn triggers a `SELECT` statement against the `Albums` table, as we see in Figure 3-7.

In this situation, I'd be *very* surprised if the overall latency of this operation didn't increase. That may not be a significant problem in this particular case, as this report is generated monthly and could therefore be aggressively cached (we'll explore this topic in more detail in "Caching"). But if this is a frequent operation, that could be more problematic. We can mitigate the likely impact of this increase in latency by allowing for SKUs to be looked up in the `Catalog` microservice in bulk, or perhaps even by caching the required album information locally.

Data Integrity

Databases can be useful in ensuring integrity of our data. Coming back to Figure 3-6, with both the `Album` and `Ledger` tables being in the same database, we could (and likely would) define a foreign key relationship between the rows in the `Ledger` table and the `Album` table. This would ensure that we'd always be able to navigate from a record in the `Ledger` table back to information about the album sold, as we wouldn't be able to delete records from the `Album` table if they were referenced in `Ledger`.

With these tables now living in different databases, we no longer have enforcement of the integrity of our data model. There is nothing to stop us from deleting a row in the `Album` table, causing an issue when we try to work out exactly what item was sold.

To an extent, you'll simply need to get used to the fact that you can no longer rely on your database to enforce the integrity of inter-entity relationships. Obviously, for data that remains inside a single database, this isn't an issue.

There are a number of work-arounds, although "coping patterns" would be a better term for ways we might deal with this problem. We could use a soft delete in the `Album` table so that we don't actually remove a record but just mark it as deleted. Another option could be to copy the name of the album into the `Ledger` table when a sale is made, but we would have to resolve how we wanted to handle synchronizing changes in the album name.

Transactions

Many of us have come to rely on the guarantees we get from managing data in transactions. Based on that certainty, we've built applications in a certain way, knowing that we can rely on the database to handle a number of things for us. Once we start splitting data across multiple databases, though, we lose the safety of the ACID transactions we are used to. (I explain the acronym ACID and discuss ACID transactions in more depth in Chapter 6.)

For people moving from a system in which all state changes could be managed in a single transactional boundary, the shift to distributed systems can be a shock, and often the reaction is to look to implement distributed transactions to regain the guarantees that ACID transactions gave us with simpler architectures. Unfortunately, as we'll cover in depth in "Database Transactions", distributed transactions are not only complex to implement, even when done well, but they also don't actually give us the same guarantees that we came to expect with more narrowly scoped database transactions.

As we go on to explore in "Sagas", there are alternative (and preferable) mechanisms to distributed transactions for managing state changes across multiple microservices, but they come with new sources of complexity. As with data integrity, we have to come to terms with the fact that by breaking apart our databases for what may be very good reasons, we will encounter a new set of problems.

Tooling

Changing databases is difficult for many reasons, one of which is that limited tools remain available to allow us to make changes easily. With code, we have refactoring tooling built into our IDEs, and we have the added benefit that the systems we are changing are fundamentally stateless. With a database, the things we are changing have state, and we also lack good refactoring-type tooling.

There are many tools out there to help you manage the process of changing the schema of a relational database, but most follow the same pattern. Each schema change is defined in a version-controlled delta script. These scripts are then run in strict order in an idempotent manner. Rails migrations work in this way, as did DBDeploy, a tool I helped create many years ago.

Nowadays I point people to either Flyway or Liquibase to achieve the same outcome, if they don't already have a tool that works in this way.

Reporting Database

As part of extracting microservices from our monolithic application, we also break apart our databases, as we want to hide access to our internal data storage. By hiding direct access to our databases, we are better able to create stable interfaces, which make independent deployability possible. Unfortunately, this causes us issues when we do have legitimate use cases for accessing data from more than one microservice, or when that data is better made available in a database, rather than via something like a REST API.

With a reporting database, we instead create a dedicated database that is designed for external access, and we make it the responsibility of the microservice to push data from internal storage to the externally accessible reporting database, as seen in Figure 3-8.

The reporting database allows us to hide internal state management, while still presenting the data in a database—something which can be very useful. For example, you might want to allow people to run off ad hoc defined SQL queries, run large-scale joins, or

make use of existing toolchains that expect to have access to a SQL endpoint. The reporting database is a nice solution to this problem.

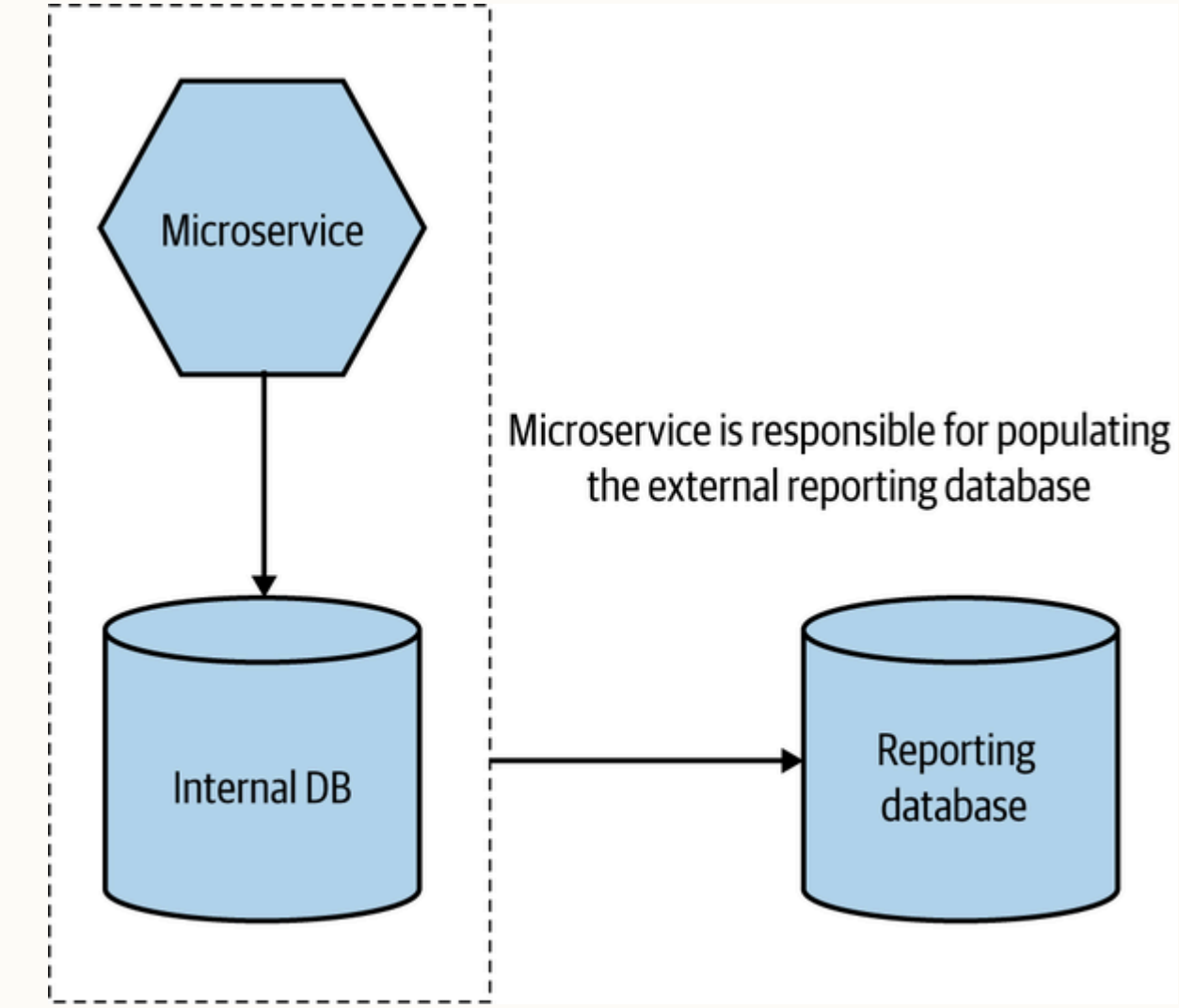


Figure 3-8. An overview of the reporting database pattern

There are two key points to highlight here. Firstly, we still want to practice information hiding. So we should expose only the bare minimum of data in the reporting database. This means that what is in the reporting database may be only a subset of the data the microservice stores. However, as this is not a direct mapping, it creates the opportunity to come up with a schema design for the reporting database that is tailored exactly to the requirements of the consumers—this could involve using a radically different schema, or perhaps even a different type of database technology altogether.

The second key point is that the reporting database should be treated like any other microservice endpoint, and it is the job of the microservice maintainer to ensure that compatibility of this endpoint is maintained even if the microservice changes its internal implementation detail. The mapping from internal state to reporting database is the responsibility of the people who develop the microservice itself.

Summary

So, to distill things down, when embarking on work to migrate functionality from a monolithic architecture to a microservice architecture, you must have a clear understanding of *what* you expect to achieve. This goal will shape how you go about the work and will also help you understand whether you're moving in the right direction.

Migration should be incremental. Make a change, roll that change out, assess it, and go again. Even the act of splitting out one microservice can itself be broken down into a series of small steps.

If you want to explore any of the concepts in this chapter in more detail, another book I have written, *Monolith to Microservices* (O'Reilly), is a deep dive into this topic.

Much of this chapter has been somewhat high level in its overview. In the next chapter, though, we're going to start getting more technical when we look at how microservices communicate with one another.

1 Sam Newman, *Monolith to Microservices* (Sebastopol: O'Reilly, 2019).

2 Pete Hodgson, "Feature Toggles (aka Feature Flags)," martinfowler.com, October 9, 2017, *https://oreil.ly/XiU2t*.

3 Obviously, this is a simplification of what a real-world system would look like. It seems reasonable, for example, that we would record how much we sold an item for in a financial ledger!

Previous chapter
‹ 2. How to Model Microservices

Next chapter
4. Microservice Communication Styles ›